

CO3 AT1

CODING CHALLENGE - VANILLA POLICY GRADIENT AND REINFORCE

CODE:

```
import gymnasium as gym
import torch
import torch.nn as nn
import torch.optim as optim
from torch.distributions import Categorical
import matplotlib.pyplot as plt
import numpy as np

# =====
# 1. POLICY NETWORK
# =====

class PolicyNetwork(nn.Module):

    def __init__(self, state_dim, action_dim):
        super().__init__()

        self.net = nn.Sequential(
            nn.Linear(state_dim, 128),
            nn.ReLU(),
            nn.Linear(128, action_dim)
        )

    def forward(self, state):
        logits = self.net(state)
        return Categorical(logits=logits)

# =====
# 2. VALUE NETWORK
# Used by Vanilla Policy Gradient with a baseline
# =====

class ValueNetwork(nn.Module):

    def __init__(self, state_dim):
        super().__init__()

        self.net = nn.Sequential(
            nn.Linear(state_dim, 128),
            nn.ReLU(),
            nn.Linear(128, 1)
        )

    def forward(self, state):
        return self.net(state).squeeze(-1)
```

```

# =====
# 3. DISCOUNTED RETURNS
# =====

def compute_returns(rewards, gamma=0.99):

    returns = []
    G = 0

    for reward in reversed(rewards):
        G = reward + gamma * G
        returns.insert(0, G)

    return torch.tensor(returns, dtype=torch.float32)

# =====
# 4. REINFORCE
#
# Uses:
#   G_t
#
# Policy loss:
#   -G_t * log(pi(a_t | s_t))
# =====

def train_reinforce(
    episodes=500,
    gamma=0.99,
    lr=0.001
):

    env = gym.make("CartPole-v1")

    state_dim = env.observation_space.shape[0]
    action_dim = env.action_space.n

    policy = PolicyNetwork(
        state_dim,
        action_dim
    )

    optimizer = optim.Adam(
        policy.parameters(),
        lr=lr
    )

    episode_rewards = []

    for episode in range(episodes):

        state, _ = env.reset()

        log_probs = []
        rewards = []

        done = False

```

while not done:

```
    state_tensor = torch.tensor(
        state,
        dtype=torch.float32
    )

    distribution = policy(state_tensor)

    action = distribution.sample()

    log_prob = distribution.log_prob(action)

    next_state, reward, terminated, truncated, _ = env.step(
        action.item()
    )

    done = terminated or truncated

    log_probs.append(log_prob)
    rewards.append(reward)

    state = next_state

# Calculate Monte-Carlo returns
returns = compute_returns(
    rewards,
    gamma
)

# Normalize returns
if len(returns) > 1:

    returns = (
        returns - returns.mean()
    ) / (
        returns.std() + 1e-8
    )

# Policy gradient loss
loss = 0

for log_prob, G in zip(
    log_probs,
    returns
):
    loss += -log_prob * G

optimizer.zero_grad()

loss.backward()

optimizer.step()

total_reward = sum(rewards)
```

```

        episode_rewards.append(total_reward)

    if episode % 50 == 0:

        print(
            f"[REINFORCE] "
            f"Episode {episode}, "
            f"Reward = {total_reward}"
        )

    env.close()

    return episode_rewards


# =====
# 5. VANILLA POLICY GRADIENT + BASELINE
#
# Uses:
#
#   Advantage =  $G_t - V(s_t)$ 
#
# Policy loss:
#
#    $-Advantage * \log(\pi(a_t | s_t))$ 
#
# Also trains a value network.
# =====

def train_vanilla_pg(
    episodes=500,
    gamma=0.99,
    policy_lr=0.001,
    value_lr=0.001
):

    env = gym.make("CartPole-v1")

    state_dim = env.observation_space.shape[0]
    action_dim = env.action_space.n

    policy = PolicyNetwork(
        state_dim,
        action_dim
    )

    value_network = ValueNetwork(
        state_dim
    )

    policy_optimizer = optim.Adam(
        policy.parameters(),
        lr=policy_lr
    )

    value_optimizer = optim.Adam(
        value_network.parameters(),

```

```

    lr=value_lr
)

episode_rewards = []

for episode in range(epochs):

    state, _ = env.reset()

    states = []
    log_probs = []
    rewards = []

    done = False

    while not done:

        state_tensor = torch.tensor(
            state,
            dtype=torch.float32
        )

        distribution = policy(
            state_tensor
        )

        action = distribution.sample()

        log_prob = distribution.log_prob(
            action
        )

        next_state, reward, terminated, truncated, _ = env.step(
            action.item()
        )

        done = terminated or truncated

        states.append(state_tensor)
        log_probs.append(log_prob)
        rewards.append(reward)

        state = next_state

    # -----
    # Calculate Monte-Carlo returns
    # -----

    returns = compute_returns(
        rewards,
        gamma
    )

    states_tensor = torch.stack(states)

    # -----
    # Estimate V(s)

```

```

# -----

values = value_network(
    states_tensor
)

# -----
# Advantage
#
#  $A(s,a) = G - V(s)$ 
# -----

advantages = returns - values.detach()

# -----
# Policy loss
# -----

policy_loss = 0

for log_prob, advantage in zip(
    log_probs,
    advantages
):

    policy_loss += (
        -log_prob * advantage
    )

# -----
# Value loss
# -----

value_loss = nn.MSELoss()(
    values,
    returns
)

# -----
# Update policy
# -----

policy_optimizer.zero_grad()

policy_loss.backward()

policy_optimizer.step()

# -----
# Update value network
# -----

value_optimizer.zero_grad()

value_loss.backward()

value_optimizer.step()

```

```

total_reward = sum(rewards)

episode_rewards.append(
    total_reward
)

if episode % 50 == 0:

    print(
        f"[VANILLA PG] "
        f"Episode {episode}, "
        f"Reward = {total_reward}"
    )

env.close()

return episode_rewards


# =====
# 6. RUN BOTH ALGORITHMS
# =====

print("\nTraining REINFORCE...")
reinforce_rewards = train_reinforce(
    episodes=500
)

print("\nTraining Vanilla Policy Gradient...")
vanilla_rewards = train_vanilla_pg(
    episodes=500
)


# =====
# 7. SMOOTH REWARD CURVES
# =====

def moving_average(values, window=20):

    return np.convolve(
        values,
        np.ones(window) / window,
        mode="valid"
    )

reinforce_smooth = moving_average(
    reinforce_rewards
)

vanilla_smooth = moving_average(
    vanilla_rewards
)

```

```

# =====
# 8. COMPARISON GRAPH
# =====

plt.figure(figsize=(12, 6))

plt.plot(
    reinforce_smooth,
    label="REINFORCE"
)

plt.plot(
    vanilla_smooth,
    label="Vanilla Policy Gradient + Baseline"
)

plt.xlabel("Episode")

plt.ylabel("Average Reward")

plt.title(
    "REINFORCE vs Vanilla Policy Gradient"
)

plt.legend()

plt.grid(True)

plt.tight_layout()

plt.show()

```

OUTPUT:

```

Training REINFORCE...
[REINFORCE] Episode 0, Reward = 12.0
[REINFORCE] Episode 50, Reward = 17.0
[REINFORCE] Episode 100, Reward = 26.0
[REINFORCE] Episode 150, Reward = 114.0
[REINFORCE] Episode 200, Reward = 54.0
[REINFORCE] Episode 250, Reward = 118.0
[REINFORCE] Episode 300, Reward = 146.0
[REINFORCE] Episode 350, Reward = 100.0
[REINFORCE] Episode 400, Reward = 290.0
[REINFORCE] Episode 450, Reward = 201.0

Training Vanilla Policy Gradient...
[VANILLA PG] Episode 0, Reward = 47.0
[VANILLA PG] Episode 50, Reward = 40.0
[VANILLA PG] Episode 100, Reward = 36.0
[VANILLA PG] Episode 150, Reward = 67.0
[VANILLA PG] Episode 200, Reward = 67.0
[VANILLA PG] Episode 250, Reward = 146.0
[VANILLA PG] Episode 300, Reward = 174.0
[VANILLA PG] Episode 350, Reward = 500.0
[VANILLA PG] Episode 400, Reward = 405.0
[VANILLA PG] Episode 450, Reward = 247.0

```

GRAPH:

